

Part 1: Understanding Hopfield Networks

Question 1.1 (8 marks): In writing, define a Hopfield network using the eight elements of the PDP framework described in week 3 of the class (see the lecture slides and the readings).

1. Set of processing units
 - a. The Hopfield network has one set of neurons.
 - b. There are not input/output neurons per se. Rather, the processing units get an input state and eventually converge to a final state.
2. State of activation at time t for each unit
 - a. The state of activation can be thought of as a “spike” (1) or “no spike” (-1).
 - b. The value of the unit is set to 1 if it is above a certain threshold or -1 if it is below a certain threshold.
3. Output function for each unit
 - a. The output function is the identity because the activation state is passed on
4. Pattern of connectivity between all of the units
 - a. The units are fully connected in this network. That is, each unit in the network is connected to each other unit in the network
 - b. These connections are bidirectional, and each connection has a weight. These connections are represented as a weight matrix where $W_{ij} = W_{ji}$. We assume no self-connection and therefore that $W_{ii} = 0$.
5. Propagation rule for sending activity between units
 - a. The propagation rule is net input. The input to a unit is the weighted sum of the outputs of all the connected.
 - b. The outputs here are the states of the unit at the time t
6. Activation rule for combining the propagated activity with current activity
 - a. The activation rule classifies the propagated activity as spike (1) if it is above a threshold, or a no-spike (-1) if it is not.
7. Learning rule
 - a. The learning rule is implements Hebbian learning through gradient descent where we minimize an energy function, not a loss or accuracy function like in assignment 1.
8. Environment that provides inputs to some subsets of the units
 - a. The environment of the Hopfield network is the energy landscape. The environment is represented as “time-varying stochastic function over the space of input patterns” and the energy function is randomly sampled to provide inputs to a unit.

Question 1.2 (2 marks): What is the energy function for a traditional Hopfield network, and why do we call it an “energy” function?

The energy function is (1). The energy function serves as the objective function in the network, the same way a loss function would be the objective in a multilayer perceptron.

$$(1) E(o(t)) = -\sum_{i,j} W_{ij} o_i(t) o_j(t) + \sum_i o_i(t) \theta_i$$

It is called the energy function because neurons in a Hopfield network mimic the spin of charged particles in the Ising model of ferromagnetism. They have positive spin (spike) and negative spin (no spike), and the particles will influence each other until they have converged to a low energy stable position.

In a Hopfield network, the low energy position is a memory and when given a stimulus, the neurons influence each other until they reach a low energy stable position (the network remembers).

Question 1.3 (2 marks): What is the partial derivative of the energy function with respect to a synaptic weight?

$$(2) \frac{\partial E(o(t))}{\partial W_{ij}} = \frac{\partial [-\sum_{k,l} W_{kl} o_k(t) o_l(t)]}{\partial W_{ij}} = \sum_{k,l} \frac{-\partial (W_{kl} o_k(t) o_l(t))}{\partial W_{ij}} = \begin{cases} -o_i(t) o_j(t) & k=i, l=j \\ 0 & k \neq i, l \neq j \end{cases}$$

Question 1.4 (4 marks): If we used the answer you gave to Question 1.3 to do gradient descent for training one pattern at a time, what type of synaptic plasticity rule is being implemented (i.e. what type of neuroscience based rule)? Explain your answer—what makes this that type of rule?

This implements Hebbian learning, commonly described by the rule 'neurons that fire together, wire together.' Specifically, if two connected neurons have the same activation (both +1 or both -1), their connection strengthens; if opposite, it weakens. However, biologically, Hebbian learning typically emphasizes positive activation (spiking) rather than strengthening connections for silent neurons, which differs slightly from this implementation.

Question 1.5 (4 marks): If you wanted to train a Hopfield network on multiple patterns in one go, what equation could you use to determine the values of the synaptic weights?

If we wanted to train multiple patterns in one go, we would use gradient descent (3) to update the weights. Unlike the outer product method for storing 1 pattern, this method allows us to do 1 weight update per pattern per epoch and prevent the last pattern from dominating.

$$(3) \Delta W_{ij} = \eta o_i(t) o_j(t)$$

$$(4) W_{ij} = \eta \sum_i^N o_i(t) o_j(t)$$

Question 1.6 (4 marks): In your own words, what do these learning rules do to the energy landscape of a Hopfield network?

The learning rules change the weights, so the stored patterns are low energy attractor states in the landscape. In other words: if pattern O^k is a stored pattern, $-\sum_{i,j} W_{ij} o_i^k(t) o_j^k(t)$ is a minimum value and if $O^{k'}$ is similar it will end up being pulled to that minima.

•Question 1.7 (6 marks): What do you think would happen in a Hopfield network if you presented the complement (i.e. the opposite) of a stored pattern to the network? Consider the equation you gave in to Questions 1.2 and 1.5, and think about what it implies for memory storage in Hopfield networks.

Presenting the complement pattern results in the same energy minima because changing the sign of all neurons simultaneously doesn't alter the product. Thus, both the original and complement patterns converge to the same memory.

$$E(o(t)) = - \sum_{i,j} W_{ij} o_i(t) o_j(t)$$
$$\Delta W_{ij} = \eta o_i(t) o_j(t)$$

Say for example changing $o_1(0)=1$ to $o_1(0)=-1$ and $o_1(0)=-1$ to $o_1(0)=1$. The values of $E(o(0))$ and ΔW_{12} would not change.

A real-life example of brains being able to converge to a memory given the compliment is using dark mode in a text editor. Both character strings below are equally recognizable to our brains.

This is a sentence This is a sentence

While our brains are more complicated than this, this is a good representation of how, in a Hopfield network, the relationships between the signs of the neurons are more important than the values in isolation.

Part 2: Designing the Hopfield Network

Question 2.1 (4 marks): What variables did you declare in your Hopfield network class? Why did you include these in the class, i.e. justify your design decisions?

In my Hopfield network I declared the following class variables:

1. `self.N`: Contains the number of neurons, used for easy reference later.
2. `self.state`: A length N vector that represents the current state of the network. Initialized to zeroes.

3. self.weights: A symmetric NxN matrix to represent the connections between the neurons in the network. I decided to make this a class variable because the weights are a property of the network and must be consistent across the class.
4. self.threshold: A length N vector that represents the threshold of each neuron. Initialized to zeroes. In feed forward networks the bias (analogy to threshold) is often just another row of the weight matrix. However, because the updates are different, I decided to have it be its own structure.
5. self.learning_rate: A floating point number that controls the magnitude of the weight updates. This was made a class variable for easy global access and because I wanted to experiment with different learning rates for different tasks. I noticed that 0.1 was too high of a learning rate and led to one pattern dominating whereas 0.0001 was too slow.
6. self.threshold_learning_rate: I noticed that threshold updates were much smaller than weight updates and required different levels of tuning to prevent all (or none) of the neurons spiking.

Question 2.2 (4 marks): First, design a one-pattern-at-a-time learning rule function. How will you calculate the weight and threshold updates for the network?

I calculate the weight and threshold updates via a gradient descent. Using (3) and (6) I calculated weight updates. Because there is only one pattern, we only need to perform a weight update once per epoch. The threshold update is performed element wise.

$$(5) \frac{\partial E(o(t))}{\partial \theta_i} = \frac{\partial (-\sum_{k,j} W_{kj} o_k(t) o_j(t) + \sum_k \theta_k o_k(t))}{\partial \theta_i} = \frac{\partial (\sum_k \theta_k o_k(t))}{\partial \theta_i} = 1 \text{ if}$$

$$(6) \Delta \theta_i = -\eta o_i(t)$$

Question 2.3 (4 marks): Now, define a mass pattern storage learning function. How will you set the synaptic weights and thresholds?

We can set the weights for the mass pattern storage in a similar manner to fitting one. Unlike single-pattern learning, here we sum the updates across all patterns within each epoch like in (4). This parallel update prevents a single pattern from dominating the learned representation and allow us to reshape the network to have multiple (hopefully) distinct minima.

Reminder:

$$(4) W_{ij} = \eta \sum_i^N o_i(t) o_j(t)$$